trackifyr – AI Based Cognitive Load Estimation via Natural Activity Monitoring

Final Year Project - Mid Report (Phase I)

Session 2022-2026

A Final Year Project-I submitted in partial fulfillment of
the requirements for the Degree

of

BS in Computer Science

Department of Computer Science

University of the Punjab, Lahore

Project Details

Project ID (for office use)	
Type of project	[✓] Traditional [ ] Industrial [ ] Continuing
Nature of project	[ ] Development [✓] Research & Development
Sustainable Development Goals(SDGs)	[✓] Good Health and Well-Being [✓] Quality Education [✓] Industry, Innovation, and Infrastructure [ ] Gender Equality [ ] Decent Work and Economic Growth [ ] Climate Action
Area of specialization	[✓] Artificial Intelligence (AI) [ ] Blockchain [ ] Cybersecurity [✓] Data Science and Analytics [ ] Game Development [ ] Internet of Things (IoT) [ ] Natural Language Processing (NLP) [ ] Mobile App Development [✓] Web Development

Project Group Members

Sr.#	Reg. #	Student Name	Email ID	Signature
(i)	Group Leader BCSF22M023	Muhammad Moin U Din	bcsf22m023@pucit.edu.pk	
(ii)	BCSF22M031	Muhammad Junaid Malik	bcsf22m031@pucit.edu.pk	
(iii)	BCSF22M043	Muhammad Subhan Ul Haq	bcsf22m043@pucit.edu.pk	

Declaration: The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.

Plagiarism Free Certificate

This is to certify that, I am Muhammad Moin U Din S/o Abdul Hameed, group leader of FYP under registration no BSCS-F22-M023 at the Computer Science Department, Punjab University, Lahore . I declare that my FYP proposal is checked by my supervisor and the similarity index is _____% that is less than 20%, an acceptable limit by HEC. The report is attached herewith as Appendix A.

Date: _____ Name of Group Leader: Muhammad Moin U Din Signature: _____

Name of Supervisor: Prof. Tayyaba Tariq Co-Supervisor (if any):_____-_____

Designation: Assistant Professor Designation: _____-_____

Signature: _____ Signature: _____-_____

HoD: _____ Signature: _____

Table of Contents

Chapter 1. Introduction 5
1.1 Introduction 5
1.2 Problem Statement 5
1.3 Proposed Solution 5
1.4 Main Objectives 6
1.5 Assumptions & Constraints 6
1.6 Project scope 6
1.7 Software Development Lifecycle Model 6
Chapter 2. Requirement Analysis 7
2.1 Literature Review 7
2.2 Research Gap Identification 7
2.3 Requirements Elicitation 8
2.3.1 Functional Requirements 8
2.3.2 Non-functional Requirements 8
2.3.3 Requirements Traceability Matrix 8
2.4 Use Case Description 9
Chapter 3. System Design 10
3.1 Activity Diagram 10
3.2 Software Architecture Diagram 11
3.3 Database Diagram (ERD-Next Phase) 11
3.4 Dataset Details 11
3.5 Algorithm / Model Selection 11
Chapter 4. Implementation 12
4.1 Work Breakdown Structure (WBS) 12
4.2 Team Roles and Responsibilities 13
4.3 Tools and Technologies 15
4.4 Implementation Details 15
4.5 Screenshots of Prototype / System 15
4.6 Challenges During Implementation 18
Chapter 5. Results & Analysis (Next Phase) 18
Chapter 6. Conclusions (Next Phase) 18
Chapter 7. References 18

List of Tables

Table 2.1 Requirements Traceability Matrix 8
Table 4.1 Team Roles and Responsibilities 13

List of Figures

Figure 3.1 Activity Diagram 10
Figure 3.2 Software Architecture Diagram 11
Figure 4.1 Gantt Chart 14
Figure 4.2 Web Dashboard 15
Figure 4.3 Web Dashboard Graphs 16
Figure 4.4 Web Dashboard Sessions & Feedback 16
Figure 4.5 Web Sign-in 16
Figure 4.6 Mouse/Keyboard Tracking 17

Chapter 1. Introduction

1.1 Introduction

As the sphere of online education, remote work, and collaboration rapidly develops, people spend more and more time before computers. Although digital wellbeing tools are becoming mainstream, the majority of them offer superficial data, including screen time and app usage. Such measures are not indicative of the real mental work or thought process of users.

Cognitive load is defined as the **mental effort** that is being utilized in working memory during the working process. Large cognitive load could signify mental exhaustion or overwork, but small cognitive load could signify disengagement or lack of concentration. Real-time cognitive load estimation can be used to enhance the productivity, learning, and mental health.

The proposed project, AI-Based Cognitive Load Estimation through Natural Activity Monitoring, will focus on developing the non-intrusive system that can estimate the cognitive load of the user based on the naturally occurring expressions of **activity** (facial expression, eye gaze, blink rate, keyboard/mouse interaction patterns). It is a system that involves the use of computer vision, behavioral analysis, and machine learning to **categorize** cognitive load into Low, Medium, and High states and display the findings on a web-based dashboard.

1.2 Problem Statement

The existing digital productivity and wellbeing solutions cannot estimate cognitive load in real time in non-intrusive ways. Older methods like self reporting are subjective and physiological sensing (EEG, ECG) methods need specialized equipment and cannot be used in real life.

The currently used behavioral observation systems tend to focus on one of the modalities: facial expression or eye contact and fail to offer any practical feedback. In addition, a high number of research solutions are restricted to the controlled laboratory environment and fail to transfer to practical use systems.

Thus, more intelligent, practical, and privacy-conscious system is required that is capable of automatically identifying cognitive load on natural computer usage and give users meaningful feedback.

1.3 Proposed Solution

The alternative solution is a multimodal artificial intelligence system that approximates the cognitive load by using visual and behavioral cues. The system consists of:

- **Activity Monitoring Module**: Records facial expressions (faces, blink, direction, expressions) with a web camera and engagement habits (typing rate, duration, mouse movement).
- **Feature Extraction Module**: Performs the processing of unprocessed data to obtain numerical features that are related to cognitive load.
- **Cognitive Load Classification Module**: This is an application that runs machine learning models like Random Forest (baseline) and CNN+LSTM (advanced) with the objective of classifying levels of loads.
- **Web Dashboard**: Shows real-time and historical data of cognitive loads in the form of visual analytics.
- **Feedback Mechanism**: Gives prompts like a break in case of cognitive overload.

1.4 Main Objectives

The key aims of the given project are:

1. **Data Collection**: To gather and process multimodal data (facial, gaze, input behaviors) of public datasets (DAISEE, CLT) and pilot studies.
2. **Feature Engineering**: To extract salient features, which are related to low, medium, and high cognitive load.
3. **Model Development**: To build ML models (Random Forest, CNN+LSTM) to achieve real-time classification.
4. **System Implementation**: To develop a working application to capture activities and a Web dashboard to view them.
5. **Validation**: To check the accuracy of the system on benchmark data and in real-time tests.
6. **Feedback Mechanism**: To deploy a smart alert system that will make actionable and timely suggestions (e.g., Take a break) depending on the identified state of cognition.
7. **Privacy**: To protect the privacy of the users, designing the system to act on the video information locally, not to store the raw video footage, and be able to offer the user control over the monitoring sessions.

1.5 Assumptions & Constraints

1.5.1 Assumptions:

- Monitoring is done on informed consent by users.
- Adequate pointers of cognitive load include webcam and interaction data.
- The initial model training may be carried out using public datasets.

1.5.2 Constraints:

- Browser systems do not have access to system-wide keyboard and mouse events.
- The deep learning models are very demanding in terms of computational power.
- The availability of data to train in the real world is limited during the FYP period.
- Raw video data storage is impeded by privacy and ethical concerns.

1.6 Project scope

The extent of the project will involve designing and developing a system which will categorize cognitive load into three levels during digital work or study session. The system is concerned with real-time monitoring, visualization and feedback. It is non-medical and non-psychological in its purpose, and is only meant to aid productivity and learning.

1.7 Software Development Lifecycle Model

An Agile SDLC is used, which is an Iterative and Incremental model. This enables the project to be created in stages, beginning with simple data capture and simple data models, with slow advancement of feature capture, model precision, and system incorporation.

Chapter 2. Requirement Analysis

2.1 Literature Review

The cognitive load estimation and engagement detection domain has passed the theoretical model to sophisticated AI-based systems. Cognitive Load Theory (CLT) introduced by **Sweller** is the cornerstone of this area of study and it determined that the cognitive load involved when solving problems has a direct effect on learning. Although this offered theoretical background it still failed to offer automated means of measurement.

Recent studies have been concerned with practical automated detection wherein three major approaches are used:

1. **Computer Vision and Facial Analysis:** The earliest systems that sought to automate engagement detection were based on Convolutional Neural Networks (CNNs) to determine facial expressions per frame. These models may be very effective in the cause of processing of still images, but may fail to paint a picture of behavior change with time. To overcome that, scientists developed Spatiotemporal Models including hybrid CNN+LSTM (Long Short-Term Memory) and EfficientNet-RNN models, which process series of frames to identify fatigue or distraction over time.Scientists tackled this challenge by developing Spatiotemporal Models. These include hybrid approaches like CNN+LSTM (Long Short-Term Memory) and EfficientNet-RNN models, which analyze a sequence of frames over time to detect fatigue or distraction.

2. **Physiological and Multimodal Monitoring:** A large part of the current studies is based on the use of physiological biomarkers, including EEG and ECG, to estimate cognitive load. An example is Strle et al. who used physiological sensors to anticipate cognitive load on the road. Equally, Liu et al. recorded good performance with Multiview Learning with physiological signals. Such approaches are however, typically invasive and limited in the lab. Anders et al. tried to gather data under uncontrolled real-world conditions in an attempt to move towards non-intrusive monitoring but didn't formulate predictive models.

3. **Behavioral and Interaction Analysis:** In the recent developments, there has been a shift towards Natural Activity Monitoring, which makes use of both visual signals and interaction patterns. The paper has examined the use of Temporal Convolutional Networks (TCNs) in anomaly detection to determine disengagement. In addition, multimodal fusion algorithms, where behavioral changes are used in conjunction with facial features such as mouse movements and eye gaze, have been demonstrated to acquire significantly better classification accuracy than unimodal methods.Significantly better classification accuracy is achieved by using multimodal fusion algorithms, which combine behavioral changes like mouse movements and eye gaze with facial features, compared to using unimodal methods alone. Significant datasets like DAISEE (affective states) and CLARE (real-time cognitive load) have been used as the benchmark when training these models.

2.2 Research Gap Identification

The research gaps that have been identified are:

- Absence of real time multimodal cognitive load estimation mechanisms.
- Minimal emphasis on practical applications that can be deployed.
- Over-dependence on invasive physiological monitoring.
- Absence of integrated feedback and visualization capabilities.

2.3 Requirements Elicitation

2.3.1 Functional Requirements

- **FR-01 Data Capture:** The system must be capable of capturing the video frames in the webcam and input events, i.e. taking the mouse coordinates and timestamps in case of key presses.
- **FR-02 Preprocessing:** The system should process the information using data cleaning to remove noise in the form of gaze signals and normalize the signal obtained.
- **FR-03 Classification:** System shall classify the user state as Low, Medium or High within every N seconds.
- **FR-04 Dashboard:** This has to show the past engagement patterns and the current status.
- **FR-05 Warning:** System should give notifications (e.g., Take a break) whenever detecting that the load is High.
- **FR-06 Session Management:** The system will enable a user to manually start and stop a monitoring session through the desktop application interface.
- **FR-07 Data Logging:** The system will store the results of classification (including timestamps) and feature vectors to either a local database or a secure cloud database in order to analyse them historically.

2.3.2 Non-functional Requirements

- **NFR-01 Privacy:** No raw video footage is to be stored; only extracted numerical features.
- **NFR-02 Latency:** Inference has to be done in near real-time (less than 1-2 seconds) to give a feedback in time.
- **NFR-03 Efficiency:** The background logger should not take up too much RAM/CPU, interfering with the regular work of the user.
- **NFR-04 Usability:** The web dashboard will be user friendly and operational through normal web browsers (Chrome, Edge, Firefox) without any extra installation of any such plugins.

2.3.3 Requirements Traceability Matrix

Table 2.1 Requirements Traceability Matrix

Req ID	Requirement Description	Mapped Objective(s) (Section 1.4)
FR-01	Get webcam video frames and keyboard/mouse events.	Obj-1: Data Collection
FR-02	Normalize input features by cleansing gaze noise.	Obj-1: Data Collection, Obj-2: Feature Engineering
FR-03	Put the user cognitive load as Low, Medium or High after some regular intervals.	Obj-3: Model Development
FR-04	The dashboard will present both current real-time status and historical engagement trends.	Obj-4: System Implementation
FR-05	Generate alert notifications when a high cognitive load is detected.	Obj-6: Feedback Mechanism
FR-06	Give users the option of manual initiation and stop of monitoring sessions.	Obj-4: System Implementation, Obj-7: Privacy & Ethics
FR-07	Cognitive load results are logged with a time stamp and feature data is analyzed.	Obj-5: Validation
NFR-01	Make sure that privacy is ensured by not storing raw video, and extract features only.	Obj-7: Privacy & Ethics
NFR-02	Perform inference with latency under 1-2 seconds	Obj-3: Model Development
NFR-03	Reduce CPU, memory of the background logger.	Obj-4: System Implementation
NFR-04	A responsive and browsing-available web interface.	Obj-4: System Implementation

2.4 Use Case Description

2.4.1 Use Case 1: Start Monitoring Session

- **Actor:** User
- **Description:** The user starts monitoring session by the desktop application by clicking on the Start option. When activated, the system starts recording multimodal data such as the facial cues of webcam and the logs of the keyboard/mouse interaction. The acquired data is real-time processed and sent to be analyzed by cognitive load. The monitoring will go on until the user requires it to stop.

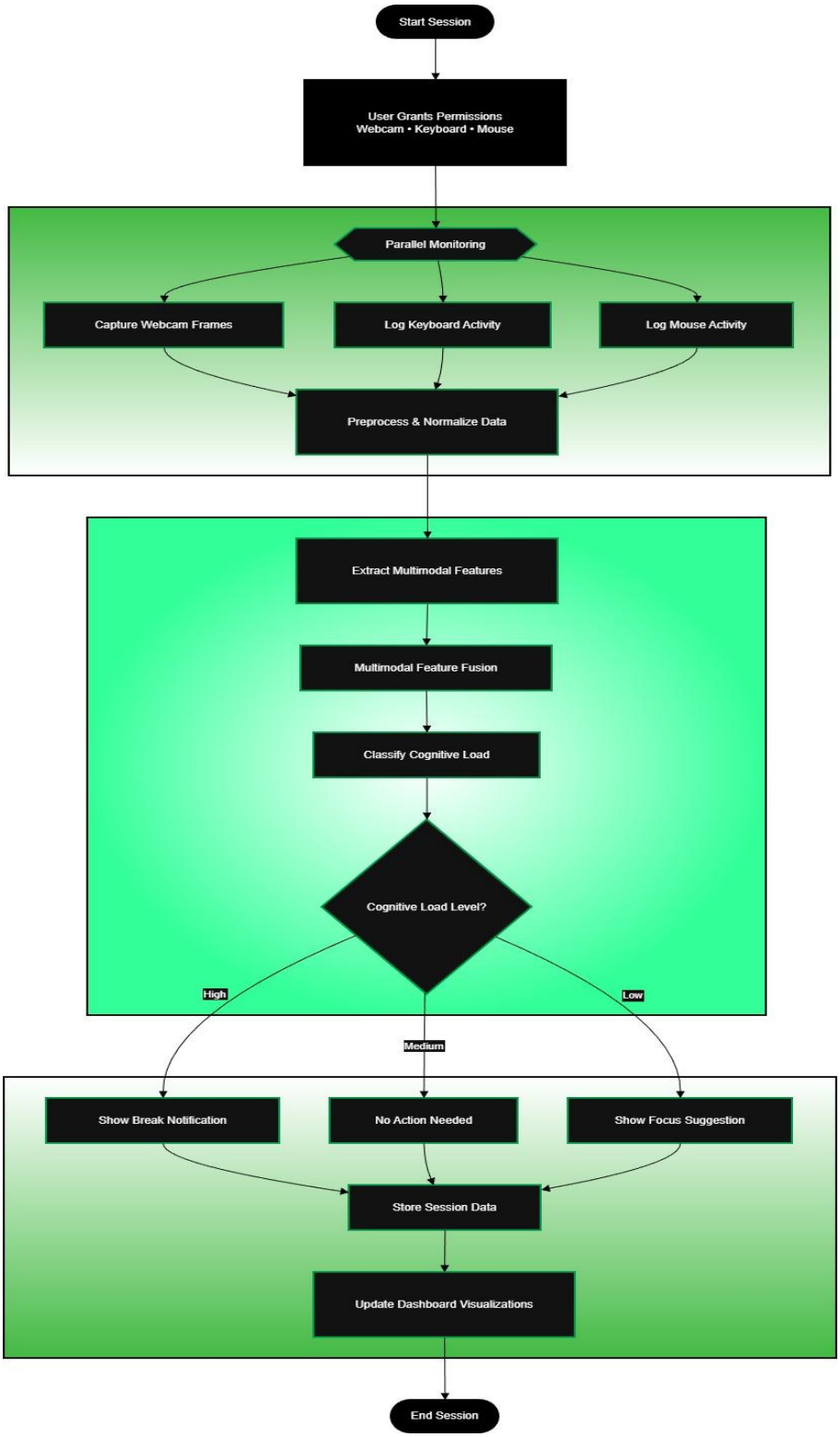
2.4.2 Use Case 2: View Analytics and Reports

- **Actor:** User
- **Description:** The user is brought to the web-based dashboard to see real-time and historical analytics of cognitive load.Upon logging in, the user is presented with a web-based dashboard that provides both real-time and historical analytics regarding cognitive load. The system retrieves the stored session data in the database and displays the engagement trends and the degree of cognitive load alongside the summary report using interactive charts and graphs to aid in the support of performance evaluation and feedback.

Chapter 3. System Design

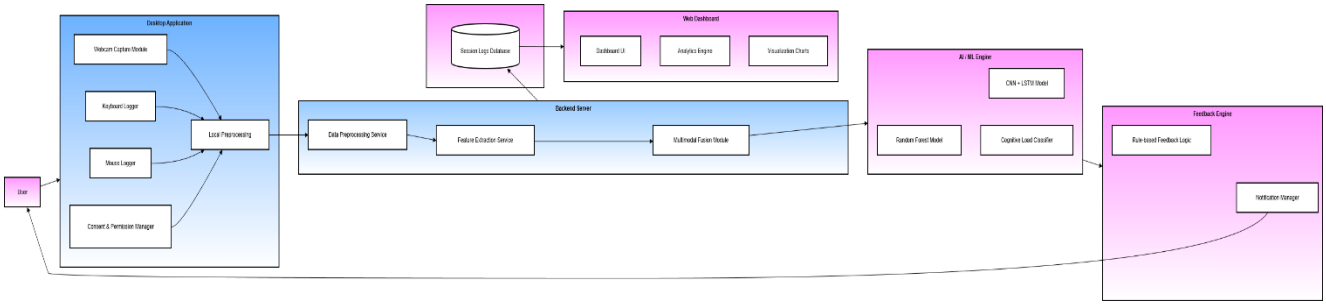
3.1 Activity Diagram

Figure 3.1 Activity Diagram



3.2 Software Architecture Diagram

Figure 3.2 Software Architecture Diagram



3.3 Database Diagram (ERD-Next Phase)

3.4 Dataset Details

- **Primary:** DAISEE (Dataset for Affective States in E-Environments) for training facial expression models.
- **Secondary:** Pilot data collected personally using the logger to capture mouse/keyboard patterns, as public datasets often lack this specific multimodal combination or **CLARE** (for real-time cognitive load).

3.5 Algorithm / Model Selection

- **Baseline:** Random Forest Classifier. The prototype was selected because of its efficiency and ability to be interpreted using tabular data (typing features).
- **Advanced:** CNN (for spatial facial features) + LSTM (for temporal sequence modeling). This shall be applied in the second phase, to work on time-dependent trends such as gradual fatigue.

Chapter 4. Implementation

4.1 Work Breakdown Structure (WBS)

4.1.1 Phase 1: Project Initialization and Setup

- Project requirement and objectives finalization.
- Choice of tools, technologies and structures.
- Development environment setup (Python, Node.js, ML libraries)
- Version control and repository setup (Git/GitHub)

4.1.2 Phase 2: Data Acquisition and Activity Logger Development

- Design of desktop based activity logger architecture
- Webcam data capture module implementation
- Keyboard and mouse interaction logging implementation
- Session control (Start/Stop monitoring)
- Local data buffering and secure transmission to backend

4.1.3 Phase 3: Backend and Data Management

- Backend API architecture design
- REST API (FastAPI/Flask) Python development
- Database schema design and integration
- Secure storage of extracted features and classification results
- API testing and validation

4.1.4 Phase 4: Machine Learning and Cognitive Load Estimation

- Face and gaze landmark detection using MediaPipe
- Feature extraction (eye aspect ratio, blink rate, typing speed, mouse dynamics)
- Training of initial ML model (Random Forest)
- The model inference is to be integrated with backend services
- Model testing based on benchmark data and pilot data

4.1.5 Phase 5: Web Dashboard and Visualization

- Design of dashboard user interface (UI/UX)
- React-based web dashboard development
- Insight into real-time and past data of cognitive load
- Back end API integration
- Dashboard testing and responsiveness validation

4.1.6 Phase 6: System Integration, Testing, and Documentation

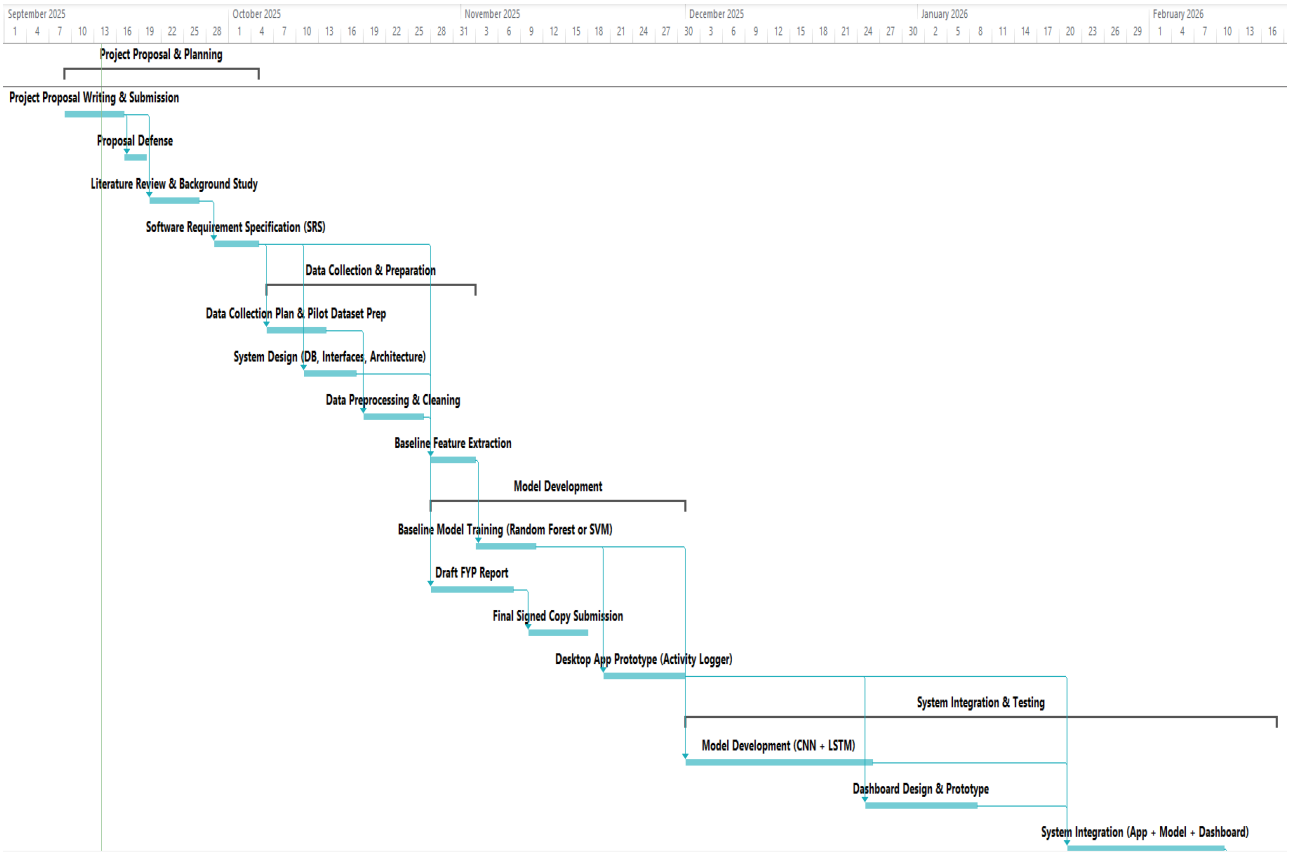
- End to end system integration
- Non-functional and functional testing
- Evaluation of performance and the use of resources
- Implementation issues identification and resolution
- Project documentation and submission of report

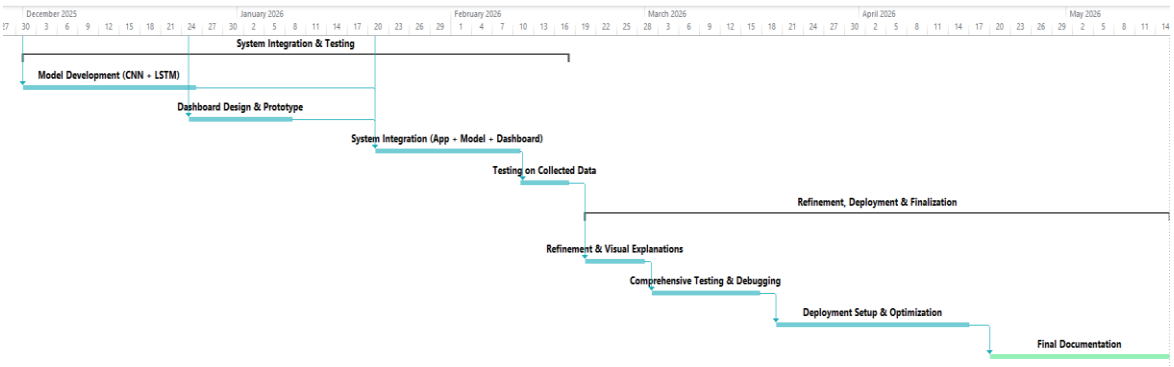
4.2 Team Roles and Responsibilities

Table 4.1 Team Roles and Responsibilities

Team Member	Activity
Moin, Junaid, Subhan	Project Proposal Writing & Submission
Moin, Junaid, Subhan	Proposal Defense
Moin, Junaid	Literature Review & Background Study
Moin	Data Collection Plan & Pilot Dataset Prep
Junaid	System Design (DB, Interfaces, Architecture)
Moin	Data Preprocessing & Cleaning
Junaid	Baseline Feature Extraction
Moin, Junaid	Baseline Model Training (Random Forest or SVM)
Subhan	Draft FYP Report
Moin, Subhan	Final Signed Copy Submission
Junaid, Subhan	Desktop App Prototype (Activity Logger)
Moin, Junaid	Model Development (CNN + LSTM)
Subhan	Dashboard Design & Prototype
Moin, Junaid, Subhan	System Integration (App + Model + Dashboard)
Moin, Junaid	Testing on Collected Data
Junaid	Refinement & Visual Explanations
Moin, Junaid	Comprehensive Testing & Debugging
Subhan	Deployment Setup & Optimization
Moin, Junaid, Subhan	Final Documentation

Figure 4.1 Gantt Chart





4.3 Tools and Technologies

- **Language:** Python (ML/Backend), JavaScript (Frontend/Logger).
- **Frameworks:** Electron.js (Desktop Logger), React.js (Web Dashboard), FastAPI (Python Backend).
- **ML Libraries:** TensorFlow/PyTorch (Model), MediaPipe (Face/Gaze), Scikit-Learn (Random Forest).
- **Database:** PostgreSQL

4.4 Implementation Details

- **Activity Logger (Electron + Python):**
  An Electron-based desktop activity logger is created to facilitate access to systems on a system-wide basis. Python library pynput is used to monitor both global events of the keyboard and a mouse that are not available to a normal web browser. The logger operates in the background and captures interaction characteristics of key sequence frequency and distance of mouse movement in active browsing.
- **Vision Module (MediaPipe):**
  The vision part is realized with the help of Google MediaPipe Face Mesh framework that recognizes 468 facial points in time. Based on such landmarks, visual characteristics that include Eye Aspect Ratio (EAR) are calculated to determine the blink rate and possible fatigue signs linked to cognitive load.
- **Web Dashboard (React):**
  React is applied to create a web-based dashboard that represents cognitive load analytics. The dashboard gets processed data in a format of JSON off the backend and displays it as interactive line charts so the user can see real-time status and historic trends of engagement.

4.5 Screenshots of Prototype / System

Figure 4.2 Web Dashboard

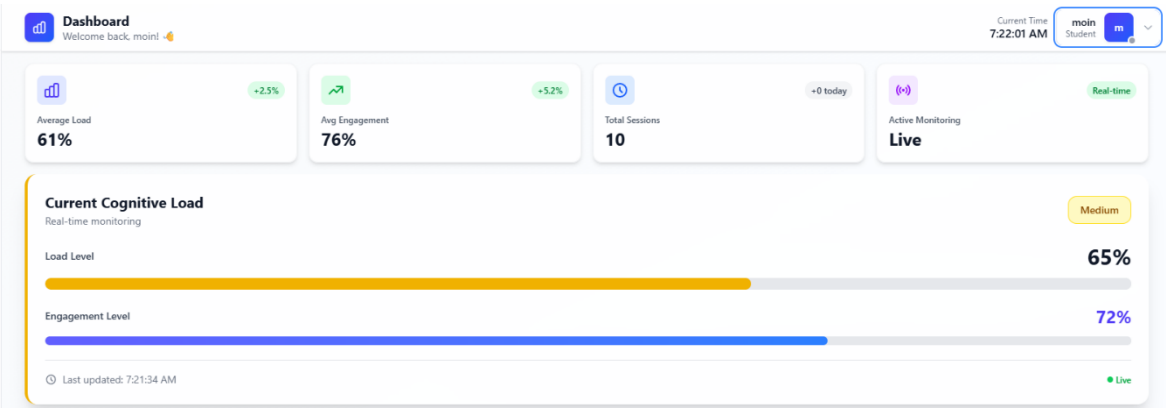


Figure 4.3 Web Dashboard Graphs

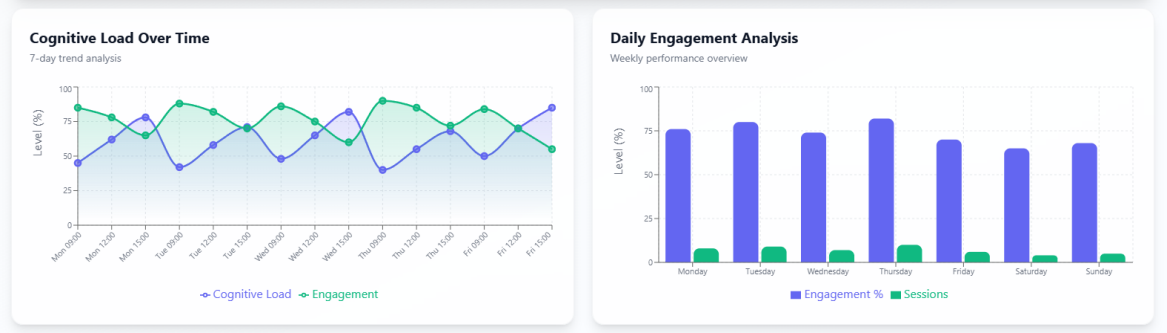


Figure 4.4 Web Dashboard Sessions & Feedback

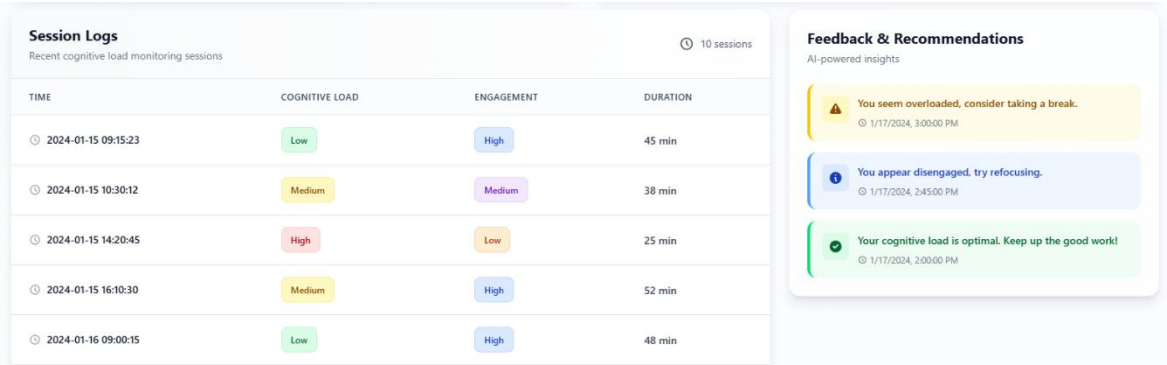


Figure 4.5 Web Sign-in

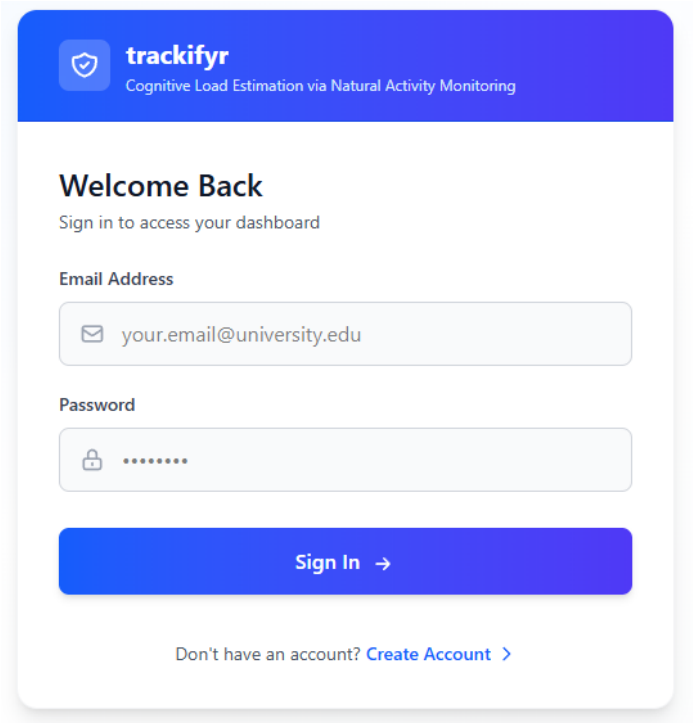


Figure 4.6 Mouse/Keyboard Tracking

```
================================================================
ACTIVITY SUMMARY - 2025-12-24 07:53:56
================================================================
Time Period: 10 seconds
Total Time: 10.00 seconds
Active Seconds: 4 seconds
Activity Percentage: 40.00%
Mouse Events: 12
Keyboard Events: 3
Total Events: 15
================================================================


================================================================
ACTIVITY SUMMARY - 2025-12-24 07:54:06
================================================================
Time Period: 10 seconds
Total Time: 10.00 seconds
Active Seconds: 10 seconds
Activity Percentage: 100.00%
Mouse Events: 59
Keyboard Events: 5
Total Events: 64
================================================================


================================================================
ACTIVITY SUMMARY - 2025-12-24 07:54:16
================================================================
Time Period: 10 seconds
Total Time: 10.00 seconds
Active Seconds: 5 seconds
Activity Percentage: 50.00%
Mouse Events: 21
Keyboard Events: 1
Total Events: 22
================================================================


================================================================
ACTIVITY SUMMARY - 2025-12-24 07:54:26
================================================================
Time Period: 10 seconds
Total Time: 10.00 seconds
Active Seconds: 7 seconds
Activity Percentage: 70.00%
Mouse Events: 29
Keyboard Events: 7
Total Events: 36
================================================================
```

4.6 Challenges During Implementation

- **Browser Sandboxing:** A pure web based solution was not able to get the global keyboard and mouse usage as the user moved between apps.
  Solution: Desktop Electron wrapper was utilized in order to allow tracking events at the system level.
- **Real-Time Latency:** Webcam video at high frame rate processing had a perceptible system lag.
  Solution: Video processing was lowered to 5 FPS and this was adequate to estimate cognitive load as well as enhance performance.
- **Privacy Concerns:** Full-time video surveillance involves the issue of user privacy.
  Solution: The system had to make the video be processed on-site and be stored with extracted numerical features rather than raw footage.
- **Limited Training Data:** There was a lack of availability of labeled multimodal datasets that comprised of facial and input behavior data.
  Solution: Pilot data were added to the solution of the problem based on public datasets during controlled testing.

Chapter 5. Results & Analysis (Next Phase)

Chapter 6. Conclusions (Next Phase)

Chapter 7. References

[1] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257-285, 1988.

[2] C. Anders, S. Moontaha, S. Real, et al., "Unobtrusive measurement of cognitive load and physiological signals in uncontrolled environments," *Scientific Data*, vol. 11, no. 2, 2024.

[3] A. Bhatti, P. Angkan, et al., "CLARE: Cognitive Load Assessment in Real-time with Multimodal Data," *arXiv preprint arXiv:2404.17098*, 2024.

[4] G. Strle, A. Košir, J. Sodnik, and K. Stojmenova, "Physiological Signals as Predictors of Cognitive Load Induced by the Type of Automotive Head-Up Display," *IEEE Access*, 2023.

[5] Y. Liu, Y. Yu, H. Tao, et al., "Cognitive Load Prediction from Multimodal Physiological Signals using Multiview Learning," *IEEE Journal of Biomedical and Health Informatics*, 2023.

[6] "TCN Autoencoder for Disengagement Detection," *IEEE Xplore*, Document ID: 9893134.

[7] "Large-Scale Engagement Dataset & Baselines," *arXiv preprint arXiv:2302.00431*, 2023.

[8] "Hybrid EfficientNet + Temporal Models for Cognitive Load," *arXiv preprint arXiv:2211.06870*, 2022.

[9] "DAISEE: Dataset for Affective States in E-learning Environments," *Thesai*, Benchmark for CNN Baseline.

[10] "Multimodal Fusion for Engagement Detection: Combining Facial and Behavioral Cues," *MDPI Sensors*, 2023.

ORIGINALITY REPORT

2%	2%	0%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

MATCH ALL SOURCES (ONLY SELECTED SOURCE PRINTED)

1%

★ repository.cuilahore.edu.pk

Internet Source

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	Off		